Qommunity

LGBTQ+ Social Media Platform

Backend Engineering Roadmap · v1.0 · 2025

1. Executive Summary

Qommunity is a production-grade social media platform designed exclusively for the LGBTQ+ community. Architecturally inspired by Instagram's backend — event-driven microservices, real-time fanout, Kafka-based pub/sub, Redis-cached feeds, and a scalable Django REST API — the platform prioritizes safety, inclusivity, and performance at scale.

This document is the single source of truth for the engineering roadmap. It defines milestones, sprints, code standards, and testing strategy across every layer of the backend.

7	8	5	4
Milestones	Django Apps	Infrastructure Services	Test Layers

2. Technology Stack

The full-stack architecture is chosen for proven scalability, developer ergonomics, and alignment with the LGBTQ+ community's privacy and safety requirements.

Layer	Technology	Purpose
Backend	Django 5.x + Django REST Framework	Production-grade Python web framework
Database	PostgreSQL 16	Primary relational DB with advanced indexing

© 2025 Qommunity. All rights reserved.

Layer	Technology	Purpose
Cache / Sessions	Redis 7	Token blacklist, rate limiting, feed cache
Message Broker	Apache Kafka + Zookeeper	Event streaming, fanout, async tasks
Task Queue	Celery + Redis broker	Async jobs: notifications, media processing
Media Storage	AWS S3 / MinIO (local)	Object storage for images & videos
Media CDN	CloudFront / Cloudflare	Edge caching for media assets
Search	PostgreSQL Full-Text / Elasticsearch	User & content search
Auth	JWT (Simple JWT) + OAuth2	Secure token-based auth
API Style	REST (DRF) → GraphQL (Strawberry)	REST now, GraphQL phase 2 for Relay
Frontend (Web)	Next.js 14 + React Query / Relay	SSR + client-side data fetching
Frontend (Mobile)	React Native + Expo	Cross-platform iOS & Android
CI/CD	GitHub Actions + Docker	Automated test, lint, deploy pipeline
Monitoring	Prometheus + Grafana + Sentry	Metrics, alerting, error tracking
Containerization	Docker + Docker Compose → K8s	Local dev → production orchestration

3. Architecture Overview

3.1 Request Lifecycle

Client → Load Balancer (Nginx) → Django (Gunicorn/Uvicorn) → Service Layer → Selectors (DB) / Cache (Redis) → Response. Long-running work is offloaded to Celery workers via Redis broker and Kafka events.

3.2 Django App Responsibilities

App	Core Responsibility	Key External Integrations
users	Auth, profiles, follows, social identity fields (pronouns, orientation, etc.)	JWT, OAuth2 (Google/Apple), Redis sessions
posts	Create/edit/delete posts, hashtags, location, visibility controls	Kafka (post.created), Celery, S3
feed	Fan-out-on-write for followers, ranked feed assembly, cache invalidation	Redis (feed cache), Kafka consumer, Celery
comments	Threaded comments, replies, moderation flags	Kafka (comment.created), WebSocket notify
likes	Like/unlike posts & comments, like counts with Redis counters	Redis (counters), Kafka, Celery
media	Upload, validate, transcode images/videos, generate thumbnails	AWS S3, FFmpeg, Celery workers

© 2025 Qommunity. All rights reserved.

App	Core Responsibility	Key External Integrations
notifications	Real-time push via WebSocket (Django Channels), in-app notification center	Redis pub/sub, WebSocket consumers, FCM
common	Shared models, utilities, base classes, custom exceptions, pagination	Internal only

3.3 Service Layer Pattern

All business logic lives in services.py. Views and serializers are thin. Selectors handle all DB reads. This makes unit testing trivial — you test services in isolation without Django's HTTP stack.

File	Role & Rules
views.py	HTTP only: parse request, call service, return response. No business logic.
serializers.py	Validate input / serialize output. Call validators.py for complex rules.
services.py	All mutations: create, update, delete. Call selectors for reads inside services.
selectors.py	All DB reads. Return QuerySets or typed objects. Never mutate.
validators.py	Pure functions: raise ValidationError. Called from services & serializers.
tasks.py	Celery async tasks. Call services — never contain business logic themselves.
events.py	Kafka event publish helpers. Called from services post-commit.
permissions.py	DRF Permission classes. No business logic — only authorization checks.

4. Milestones & Roadmap

The roadmap is divided into 7 major milestones. Each milestone produces working, tested, deployable software — never half-finished features.

M#	Milestone	Focus Area	Timeline	Status
M1	**Foundation & Core Infrastructure**	Django setup, DB, Redis, Kafka, Core layer	Week 1–2	In Progress
M2	**User Identity & Authentication**	Custom User model, JWT, OAuth2, profiles	Week 3–4	Planned
M3	**Posts & Media Pipeline**	Post CRUD, S3 upload, FFmpeg transcoding	Week 5–7	Planned
M4	**Social Graph & Feed Engine**	Follows, fan-out, Redis feed cache, ranking	Week 8–10	Planned
M5	**Engagement: Likes & Comments**	Likes, comment threads, Redis counters	Week 11–12	Planned
M6	**Notifications & Real-Time**	WebSocket, Django Channels, Kafka events	Week 13–14	Planned

© 2025 Qommunity. All rights reserved.

M#	Milestone	Focus Area	Timeline	Status
M7	**Production Hardening & Launch**	Security, monitoring, perf tuning, CI/CD	Week 15–18	Planned

Milestone 1 — Foundation & Core Infrastructure (Week 1–2)

Goal: All shared infrastructure is production-ready. Every subsequent milestone builds on this stable base.

M1: Foundation & Core Infrastructure	Week 1–2	Priority
Django project structure — all 8 apps registered, settings split (base/dev/test/prod)	Week 1	**Critical**
core/database: BaseModel (UUID PK, timestamps), SoftDeleteMixin, TransactionMixin	Week 1	**Critical**
core/exceptions: BaseAPIException, error_codes.py, consistent JSON error envelope	Week 1	**Critical**
core/redis: RedisClient singleton, cache.py helpers, rate_limit.py (token bucket)	Week 1	**Critical**
core/kafka: KafkaProducer, KafkaConsumer base classes, topics.py registry	Week 1	**Critical**
core/middleware: ExceptionHandlerMiddleware, RequestLoggingMiddleware, MetricsMiddleware	Week 2	**High**
core/pagination: CursorPagination (Instagram-style) + Connection for Relay	Week 2	**High**
core/security: JWT utils, hashing.py (bcrypt), app_version.py header check	Week 2	**High**
config/settings: DATABASE, REDIS, KAFKA, CELERY, CORS, LOGGING fully configured	Week 2	**Critical**
Docker Compose: verify all 5 services healthy, add healthcheck endpoints	Week 1	**Critical**
Write tests: test_base_model, test_redis_client, test_rate_limiter, test_exception_handler	Week 2	**High**
pyproject.toml: ruff (linting), black (format), pytest-django, factory-boy, faker configured	Week 1	**High**

Milestone 2 — User Identity & Authentication (Week 3–4)

Goal: Secure, LGBTQ+-aware user system with custom fields (pronouns, sexual orientation, gender identity), JWT auth, OAuth2 social login, and full test coverage.

M2: User Identity & Authentication	Week 3–4	Priority
Custom User model: extend AbstractBaseUser, UserManager, REQUIRED_FIELDS	Week 3	**Critical**

© 2025 Qommunity. All rights reserved.

M2: User Identity & Authentication	Week 3–4	Priority
LGBTQ+ identity fields: pronouns, gender_identity, sexual_orientation, visibility settings	Week 3	Critical
JWT: access token (15min) + refresh token (30d), blacklist on logout via Redis	Week 3	Critical
OAuth2: Google Sign-In + Apple Sign-In flows with social_django or authlib	Week 3	High
Email verification: OTP via Celery + SendGrid, resend endpoint with rate limit	Week 3	High
Password: bcrypt hashing, forgot-password flow, min-strength validator	Week 3	Critical
User profile: bio, avatar upload stub (wired to media app later), website, private/public	Week 4	High
Follow system: Follow model, follow/unfollow service, follower/following selectors	Week 4	Critical
Block & Mute: BlockedUser, MutedUser models, enforce in feed/post selectors	Week 4	High
Safety: account_privacy (public/private/close_friends), report_user service	Week 4	High
Permissions: IsOwner, IsPrivateProfile, IsVerified permission classes	Week 4	High
Tests: test_models, test_services (auth, follow, block), test_queries, test_mutations (API)	Week 4	Critical

▸ Milestone 3 — Posts & Media Pipeline (Week 5–7)

Goal: Full post lifecycle with rich media support. Images and videos upload to S3, are transcoded asynchronously via Celery/FFmpeg, and are served through CDN.

M3: Posts & Media Pipeline	Week 5–7	Priority
Post model: content, location, hashtags (M2M), visibility, allow_comments, media FK	Week 5	Critical
Post CRUD services: create_post, update_post, soft_delete_post with Kafka events	Week 5	Critical
Hashtag model + extractor: parse #tags from content, upsert, trending counter in Redis	Week 5	High
Media model: file_type, original_url, processed_url, thumbnail_url, status enum	Week 5	Critical
S3 upload: presigned URL generation, direct-upload flow, size/type validation	Week 6	Critical
Media service: create_media_record, process_media (Celery task), update_status	Week 6	Critical

© 2025 Qommunity. All rights reserved.

M3: Posts & Media Pipeline	Week 5–7	Priority
Transcoder: FFmpeg wrapper — resize images to 1080p, compress video to H.264/AAC	Week 6	High
Thumbnail generation: extract first frame for video, resize for image posts	Week 6	High
storage.py: S3Storage class, CDN URL builder, cleanup on post delete	Week 6	High
Post visibility: public, followers_only, close_friends, private — enforced in selectors	Week 7	Critical
Post search: full-text search on content + hashtags via PostgreSQL tsvector	Week 7	Medium
Kafka event: post.created → feed fanout consumer, notification consumer	Week 7	Critical
Tests: test_models, test_media_upload (mocked S3), test_transcoder, test_services	Week 7	Critical

Milestone 4 — Social Graph & Feed Engine (Week 8–10)

Goal: Instagram-style fan-out-on-write feed with Redis caching, smart ranking, and pagination. Handles both low-cardinality (regular users) and high-cardinality (celebrity) followers efficiently.

M4: Social Graph & Feed Engine	Week 8–10	Priority
FeedItem model: user_id, post_id, score, source (follow/hashtag/explore), timestamp	Week 8	Critical
fan-out service: on post.created event, push to follower feed lists in Redis (ZSet)	Week 8	Critical
Celebrity mode: users with >10k followers use fan-out-on-read to avoid Redis explosion	Week 8	High
Feed selector: merge Redis feed with DB fallback, cursor-paginated response	Week 9	Critical
ranking.py: score = recency + engagement_rate + relationship_strength formula	Week 9	High
cache.py: feed TTL strategy, invalidation on unfollow, cache warm-up on login	Week 9	High
Explore feed: trending posts by hashtag, location, community tags — Redis sorted sets	Week 9	High
Hashtag feed: subscribe to hashtag, fetch posts sorted by recency + engagement	Week 10	Medium
Kafka consumer: feed.consumer listens to post.created, post.deleted, user.followed	Week 10	Critical
Celery fanout task: async bulk push to follower feeds (batched for performance)	Week 10	Critical

© 2025 Qommunity. All rights reserved.

M4: Social Graph & Feed Engine	Week 8–10	Priority
Tests: test fan-out logic, test cache invalidation, test ranking scores, test pagination	Week 10	Critical
Load test: simulate 1k concurrent feed reads with Locust, tune Redis pool settings	Week 10	High

Milestone 5 — Engagement: Likes & Comments (Week 11–12)

Goal: High-performance like/comment system with Redis atomic counters, threaded replies, and moderation tools.

M5: Engagement: Likes & Comments	Week 11–12	Priority
Like model: GenericForeignKey (like post OR comment), unique_together constraint	Week 11	Critical
like/unlike service: toggle with Redis INCR/DECR for fast counters, DB sync via Celery	Week 11	Critical
Like count: Redis counter as source of truth, DB reconciliation every 5 min	Week 11	High
Comment model: parent FK (threaded), depth limit (3 levels), soft delete	Week 11	Critical
Comment CRUD service: create_comment, update_comment, delete_comment, pin_comment	Week 11	Critical
Comment pagination: load top-level then lazy-load replies (cursor-paginated)	Week 12	High
Moderation: hide_comment (author/post owner), report_comment service, auto-flag keywords	Week 12	High
Kafka events: like.created → notification trigger, comment.created → notification trigger	Week 12	Critical
Permissions: CanLike, CanComment, CanModerate permission classes	Week 12	High
Tests: test_like_toggle (Redis mock), test_comment_thread, test_moderation, test_counts	Week 12	Critical

Milestone 6 — Notifications & Real-Time (Week 13–14)

Goal: Real-time WebSocket notifications via Django Channels, Kafka-driven async notification dispatch, and push notifications via FCM for mobile.

M6: Notifications & Real-Time	Week 13–14	Priority
Notification model: recipient, actor, verb, target (generic FK), read/unread, created_at	Week 13	Critical

© 2025 Qommunity. All rights reserved.

M6: Notifications & Real-Time	Week 13–14	Priority
Django Channels: ASGI setup, channel layers via Redis, routing config	Week 13	Critical
consumers.py: NotificationConsumer (WebSocket), authenticate via JWT in scope	Week 13	Critical
dispatchers.py: dispatch_notification service, select channel by notification type	Week 13	High
Kafka consumer: notification.consumer listens to like.created, comment.created, user.followed	Week 13	Critical
In-app notification center: list, mark_read, mark_all_read, delete endpoints	Week 14	High
Notification preferences: per-type toggles (email, push, in-app) stored per user	Week 14	High
FCM push: send_push_notification Celery task, device token management endpoints	Week 14	High
Email notifications: async Celery tasks via SendGrid for follows, mentions	Week 14	Medium
Tests: test_websocket_consumer (channels test client), test_notification_dispatch, test_kafka_consumer	Week 14	Critical

▸ Milestone 7 — Production Hardening & Launch (Week 15–18)

Goal: The platform is secure, observable, performant, and CI/CD-deployed. Ready for first real users.

M7: Production Hardening & Launch	Week 15–18	Priority
Security audit: OWASP Top-10 review, SQL injection scan, XSS, CSRF, rate limit audit	Week 15	Critical
Content moderation: hate speech detector, image NSFW classifier integration	Week 15	Critical
LGBTQ+ safety features: blocking propagation, outing prevention, safe messaging	Week 15	Critical
API versioning: /api/v1/ prefix, deprecation header strategy	Week 15	High
Prometheus metrics: request latency, DB query count, Kafka lag, Redis hit rate	Week 16	High
Grafana dashboards: per-endpoint SLOs, Celery queue depth, feed latency P95	Week 16	High
Sentry: error tracking, performance tracing, release tracking	Week 16	High
GitHub Actions CI: test → lint → build → push Docker image → deploy	Week 16	Critical
DB: connection pooling (pgBouncer), query plan analysis, slow query alerts	Week 17	High

© 2025 Qommunity. All rights reserved.

M7: Production Hardening & Launch	Week 15–18	Priority
Redis: cluster mode config, key expiry audit, memory-max-policy noeviction for critical keys	Week 17	High
Load testing: Locust scenarios for 10k concurrent users, post/feed/like flows	Week 17	High
Documentation: OpenAPI schema via drf-spectacular, README, CONTRIBUTING.md	Week 18	Medium
Data privacy: GDPR-compliant data export, account deletion cascade service	Week 18	Critical
Staged rollout: feature flags via Redis, canary deploy strategy	Week 18	High

5. Sprint Plan

Each sprint is 1 week. Every sprint ends with working, tested, PR-reviewed code merged to main. No half-baked features carry over.

Sprint	Focus	Deliverables	Duration
Sprint 1	**Project Bootstrap**	Folder structure, Docker healthy, base settings, pyproject.toml with all dev tools	1 wk
Sprint 2	**Core Layer**	BaseModel, exceptions, Redis client, Kafka base classes, middleware, pagination	1 wk
Sprint 3	**User Auth**	Custom User model, JWT auth endpoints, email verify, LGBTQ+ identity fields	1 wk
Sprint 4	**Social Graph**	Follow/unfollow, block/mute, user search, profile endpoints, permissions	1 wk
Sprint 5	**Posts Core**	Post CRUD, hashtags, visibility, Kafka post.created event, post search	1 wk
Sprint 6	**Media Pipeline**	S3 presigned upload, FFmpeg transcoder, thumbnail generation, CDN URLs	1 wk
Sprint 7	**Feed Engine**	Fan-out service, Redis ZSet feed, feed selector, cursor pagination	1 wk
Sprint 8	**Feed Optimization**	Celebrity mode, ranking algorithm, explore feed, hashtag feed, Kafka consumer	1 wk
Sprint 9	**Likes System**	Like/unlike toggle, Redis counters, like count API, Kafka like.created	1 wk
Sprint 10	**Comments System**	Threaded comments, CRUD, pin, moderation, comment pagination	1 wk
Sprint 11	**WebSocket & Channels**	Django Channels ASGI, NotificationConsumer, JWT auth in WebSocket	1 wk
Sprint 12	**Notification System**	Kafka notification consumer, dispatchers, FCM push, in-app notification center	1 wk

© 2025 Qommunity. All rights reserved.

Sprint	Focus	Deliverables	Duration
Sprint 13	**Security Hardening**	OWASP audit, content moderation, LGBTQ+ safety features, rate limiting review	1 wk
Sprint 14	**Observability**	Prometheus, Grafana, Sentry, structured logging, CI/CD pipeline	1 wk
Sprint 15	**Performance & DB**	pgBouncer, query optimization, Redis tuning, load tests with Locust	1 wk
Sprint 16	**Docs & Privacy**	OpenAPI schema, GDPR export/delete, feature flags, staged rollout	1 wk
Sprint 17	**Beta Stabilization**	Bug fixes from beta feedback, performance regression tests, final security scan	1 wk
Sprint 18	**Production Launch**	Go-live checklist, runbook, on-call setup, monitoring alerts tuned	1 wk

6. Testing Strategy

Every app has a tests/ directory with four test files. Coverage target: 90%+ on services and selectors, 80%+ overall. Tests are fast, isolated, and runnable in CI without external services.

6.1 Test Layers

File	What It Tests	Tools & Approach	Speed
test_models.py	Model fields, constraints, methods, signals, __str__	pytest-django, factories, in-memory SQLite or --keepdb	Fast (< 1s each)
test_services.py	Business logic in services.py, error paths, edge cases	pytest, factory_boy, mock external calls (Redis, Kafka, S3)	Fast (mocked I/O)
test_queries.py	Selectors return correct QuerySets, filtering, ordering, pagination	pytest-django, TestCase with real DB, factories for fixture data	Medium
test_mutations.py	API endpoints: auth, input validation, response shape, permissions, error codes	DRF APIClient, JWT token fixtures, parametrize happy/sad paths	Medium

6.2 Testing Conventions

- Use factory_boy for all test data — never raw model creation or fixtures JSON
- Mock Redis (fakeredis), Kafka (unittest.mock), S3 (moto) — tests must run offline
- Use @pytest.mark.django_db(transaction=True) only when testing signals or atomic blocks
- Parametrize edge cases: empty inputs, max-length strings, invalid UUIDs, unauthorized users
- Each test follows Arrange → Act → Assert with clear naming: test_create_post_fails_when_content_too_long
- Celery tasks: test with task.apply() in CELERY_TASK_ALWAYS_EAGER=True mode
- WebSocket: use channels.testing.WebsocketCommunicator for consumer tests

© 2025 Qommunity. All rights reserved.

- Load tests: Locust scenarios in tests/load/ — separate from unit tests, run in CI stage

6.3 CI Test Pipeline

Stage	Command	Passes When
Lint	ruff check . && black --check .	Zero lint errors, consistent formatting
Type Check	mypy backend/	No type errors in services.py and selectors.py
Unit Tests	pytest apps/ -m 'not integration' --cov	All pass, coverage ≥ 90% on services
Integration	pytest apps/ -m integration --reuse-db	API round-trips pass with real Postgres + Redis
Security	bandit -r apps/ && safety check	No high-severity vulnerabilities
Build	docker build -t qommunity-api .	Image builds successfully

7. Code Standards & Conventions

7.1 Python / Django

- Python 3.12+, type hints on all service and selector function signatures
- Services return typed dataclasses or model instances — never raw dicts
- Never use .filter() in views or serializers — always call a selector
- Database queries: always select_related / prefetch_related to avoid N+1
- Use @transaction.atomic in services that write to multiple models
- All API errors return: { error_code: string, message: string, details: object }
- Logging: structured JSON logs via python-json-logger, include request_id from middleware

7.2 API Design

- Base URL: /api/v1/{app}/{resource}/
- Use POST for mutations (not PUT for partial updates — use PATCH)
- Pagination: all list endpoints use cursor pagination, return next/previous cursor tokens
- Timestamps: always ISO 8601 UTC in responses
- Soft deletes: deleted objects return 404 to consumers, preserved in DB for audit
- File uploads: presigned S3 URL pattern — backend issues URL, client uploads directly

7.3 Git Workflow

- Branch naming: feature/M1-base-model, fix/M2-jwt-refresh-bug, chore/update-deps
- PR must pass all CI checks and have ≥ 1 review before merge to main
- Commit style: Conventional Commits — feat:, fix:, refactor:, test:, chore:
- No direct commits to main — always via PR
- Migrations: always squash before a milestone is marked done

© 2025 Qommunity. All rights reserved.

8. LGBTQ+ Community-Specific Features

These features differentiate Qommunity from generic social platforms and must be treated as first-class engineering concerns, not afterthoughts.

Feature	Implementation	Milestone
Pronoun & identity fields	User model: pronouns[], gender_identity, sexual_orientation with visibility per-field	M2
Safe messaging mode	Auto-blur images from non-followers, message request system before DMs	M2 / M6
Outing prevention	Identity fields hidden to non-followers by default, audit log of who viewed	M2
Community spaces	Hashtag-based community hubs, moderator roles, community-specific feed	M4
Inclusive content moderation	Keyword blocklist tuned for anti-LGBTQ+ hate speech, ML classifier integration	M7
Privacy-first defaults	New accounts default to followers_only, location sharing opt-in only	M2
Crisis resources	Keyword trigger → surface mental health resources in feed (Trevor Project, etc.)	M7
Anonymous posting	Optional anonymous mode for sensitive posts (no identity attached)	M3

9. Immediate Next Steps (Start Today)

The Docker containers are running. Migrations are applied. Here is exactly what to implement first:

1. Set up core/database/base_model.py — UUID primary key, TimestampMixin, SoftDeleteMixin
2. Set up core/exceptions/base.py — BaseAPIException with error_code + HTTP status mapping
3. Set up core/redis/client.py — Redis singleton using django-redis, test connection on startup
4. Set up core/kafka/producer.py — KafkaProducer base class wrapping confluent-kafka-python
5. Configure config/settings/testing.py — use SQLite, fakeredis, mock Kafka, CELERY_TASK_ALWAYS_EAGER
6. Write first test: test_base_model.py — verify UUID pk, timestamps auto-set, soft delete hides row
7. Wire up core/middleware/exception_handler.py — catch all exceptions, return JSON envelope
8. Add pyproject.toml entries: ruff rules, pytest settings, coverage thresholds, mypy config

Once these 8 items are done, every subsequent milestone has a solid, testable foundation to build on.

© 2025 Qommunity. All rights reserved.

Built with pride. Engineered for safety.

© 2025 Qommunity. All rights reserved.